Relearn fails on Apply – the fix

Creating a deviation now throws *"Cannot handle relearn request: health configuration has been changed"* and the deviation is not saved. Three small edits, all in `HealthCommonBusiness`. Nothing else changes.

What is happening

Apply does two things in order: it creates a new configuration version, then it saves the deviations. By the time the relearn is launched, the configuration the deviation points at is already the *previous* version.

The server checks for exactly this. `HealthCommonService` line 284:

```
      if (!activeThresholdIds.containsAll(learnedThresholdIds)) {
         throw new ConcurrentAccessException("Cannot handle relearn request: health configuration has been changed.");
      }
```

and `activeThresholdIds` is built only from the latest configuration:

```
   private Set<String> getActiveThresholdIds(String aircraftId) throws EntityNotFoundException {
      for (HealthMechanicalComponentConfiguration mcc : getLatestConfiguration(aircraftId)
            .getMechanicalComponentConfigurations().values()) {
         ...
               result.add(tc.getId());
```

So the ids we send are real, but they belong to the version that has just been replaced. The server is right to refuse them.

This is the same trap as the original green-row problem: `indicator_configuration.id` changes on every Apply, `indicator.id` does not.

The fix

Resolve the thresholds from the **latest** configuration instead of from the deviation's own indicator configuration. `getIndicatorLatestLearnedThresholds` already exists on this interface and already reads through `getLatestConfiguration`, so its ids will always pass the server's check.

Edit 1 — the helper takes ids, not a configuration

`mgs-business/.../business/health/impl/HealthCommonBusiness.java`

FIND the whole `getLearnedThresholdIds` method and REPLACE it with:

```
   private List<String> getLearnedThresholdIds(String aircraftId, String indicatorId, String severity,
         String deviatedType) throws EntityNotFoundException {
      List<String> ids = new ArrayList<>();
      if (deviatedType != null && !ThresholdType.LEARNED.name().equals(deviatedType)) {
         // The deviation turns the threshold into a fixed one: nothing to learn
         return ids;
      }
      // BCR-6605 read from the LATEST configuration. Apply has just created a new version and the
      // deviation's own indicator configuration still points at the previous one; the server
      // checks every id against the latest configuration and rejects anything older.
      for (LearnedThresholdConfiguration threshold : getIndicatorLatestLearnedThresholds(aircraftId, indicatorId)) {
         if (severity != null && !severity.equals(threshold.getThreshold().getSeverity().name())) {
            continue;
         }
         ids.add(threshold.getId());
      }
      return ids;
   }
```

The `instanceof LearnedThresholdConfiguration` test is gone because the method it now calls returns learned configurations only.

Edit 2 — the create path

`mgs-business/.../business/health/impl/HealthCommonBusiness.java`

In `createDeviations`, FIND:

```
         launchRelearn(aircraftId, loginName,
               getLearnedThresholdIds(indicatorConfiguration, dto.getSeverity(),
                     dto.getThresholdType()),
               RELEARN_DESCRIPTION_CREATE + dto.getTeReference());
```

REPLACE WITH:

```
         launchRelearn(aircraftId, loginName,
               getLearnedThresholdIds(aircraftId, indicatorConfiguration.getIndicator().getId(),
                     dto.getSeverity(), dto.getThresholdType()),
               RELEARN_DESCRIPTION_CREATE + dto.getTeReference());
```

The indicator id comes off the same `indicatorConfiguration` local that is already there, so nothing else in the method moves. The indicator id is stable across configuration versions, which is the whole point.

Edit 3 — the delete path

`mgs-business/.../business/health/impl/HealthCommonBusiness.java`

In `acknowledgeDeviations`, FIND:

```
launchRelearn(aircraftId, loginName,
      getLearnedThresholdIds(deviation.getIndicatorConfiguration(),
            deviation.getSeverity(), null),
      RELEARN_DESCRIPTION_DELETE + deviation.getTeReference());
```

REPLACE WITH:

```
launchRelearn(aircraftId, loginName,
      getLearnedThresholdIds(aircraftId,
            deviation.getIndicatorConfiguration().getIndicator().getId(),
            deviation.getSeverity(), null),
      RELEARN_DESCRIPTION_DELETE + deviation.getTeReference());
```

Test

#	Action	Expect
1	Rebuild mgs-business and the client, redeploy	–
2	Deviate a LEARNED threshold, add a TE, Apply and OK	No exception in the log, row turns green
3	`SELECT * FROM public.indicator_deviation ORDER BY start_date DESC LIMIT 5;`	The deviation is there – before this fix it was rolled back
4	`SELECT * FROM public.relearn_threshold_request ORDER BY startdate DESC LIMIT 5;`	A new row, handled = false, history_learning_time = 0
5	Deviate a FIXED threshold, leaving the type alone	No new relearn row, no exception

Step 3 matters as much as step 4. The exception aborted the whole call, so until now the deviation itself was being lost, not just the relearn.

One thing this does not fix

`getIndicatorLatestLearnedThresholds` returns learned configurations only, so phase C – a deviation that turns a FIXED threshold into a learned one – cannot use it. That case needs a sibling method on `HealthDao` returning every `ThresholdConfiguration` of an indicator from the latest configuration. It does not block anything here.